Stock Market Momentum Classifier

UI Layer (React): A React application that allows users to input a stock ticker and view a financial dashboard. It displays an interactive candlestick chart of historical stock data alongside computed technical indicators like Moving Averages (5-day, 10-day) and RSI. A prediction panel shows the C model's output (UP/DOWN for the next trading day), confidence percentage, and a historical confusion matrix evaluating the model's past performance.

Algorithm Layer (C): A custom machine learning pipeline is written in C to perform feature engineering and train a Logistic Regression model for binary classification. `StockRecord` and `ModelWeights` structs store daily market data and regression parameters. The codebase is organized into modular `.c` files: `features.c` (calculates moving averages and RSI), `logistic.c` (implements the gradient descent and sigmoid function), and `metrics.c` (calculates precision, recall, and confusion matrix), with corresponding header files (`features.h`, `logistic.h`, `metrics.h`) defining the public APIs. C threads are used to parallelize the feature engineering phase by dividing the historical time-series data into chunks, allowing simultaneous calculation of different technical indicators across threads. A Makefile compiles the project into debug and release binaries.

Communication Layer (Python): Python (Flask or FastAPI) acts as the bridge between the UI and the C algorithms. It uses the `yfinance` library to fetch raw historical stock data (Open, High, Low, Close, Volume) based on the user's ticker input. Python formats this data and invokes the compiled C binary using the `subprocess` module. After the C program engineers features and completes the model training/evaluation, Python reads the resulting predictions and metrics (via standard output or intermediate files) and serves them back to the React UI in JSON format.

Backend Layer: A lightweight storage mechanism (SQLite or CSV/JSON files) stores downloaded historical stock datasets and the trained Logistic Regression model weights. This allows the application to quickly reload previous prediction sessions without needing to re-fetch data or retrain the model from scratch.

Topics Used: Structures (`StockRecord`, `ModelWeights`), functions (`calculate_rsi`, gradient `_descent`, `compute_metrics`), pointers (dynamic arrays for stock data and weight vectors), Makefile (build automation), modular programming with custom header files (`features.h`, `logistic.h`, `metrics.h`), and C threads (parallel feature extraction across historical data chunks).